

Validación y lectura correcta de resultados

1. Por qué no basta con una accuracy alta

Un resultado alto en split aleatorio puede ocultar problemas de generalización. Por eso el proyecto incluye varias validaciones con funciones distintas.

2. Check A (evaluación directa)

Archivo: `src/validate_checks.py` → `check_a_direct_eval`

- compara `model.predict(X_test[i])` con `y_test[i]`
- evita depender de mecánicas internas del entorno para computar métrica

Resultado histórico (artefacto comprometido):

- `accuracy` ≈ 0.9939

3. Check B (anti-leakage con etiquetas barajadas)

Archivo: `src/validate_checks.py` → `check_b_shuffled_labels`

- baraja `y_train`
- reentrena brevemente
- espera rendimiento cercano a baseline de clase mayoritaria

Resultado histórico comprometido:

- `shuffled_accuracy` = 0.4773
- `leakage_detected` = false

4. Check C (split duro por CSV/día)

Archivo: `src/validate_checks.py` → `check_c_csv_split`

- train en días/patrones distintos de test
- prueba de generalización más realista

Resultado histórico comprometido:

- $\text{accuracy} \approx 0.84135$
- recall ataque y F1 bajan de forma notable frente a random split

Lectura correcta: el pipeline funciona muy bien in-distribution, pero la generalización dura es sustancialmente más difícil.

5. Leave-one-exact-CSV-out

Archivo: `src/validate_leave_one_csv_out.py`

- holdout de un CSV exacto por fold
- agrega métricas por fold y globales
- incluye métricas operativas (`balanced_accuracy`, `fpr`, `fnr`, `block_rate`, `reward_per_sample`, tiempos)

Estado a defender con rigor:

- workflow implementado en código
- sin artefacto agregado completo comprometido aún

6. Cómo explicar contradicciones aparentes en métricas

No hay contradicción entre “99.8% en random split” y “84% en split duro”. Son preguntas distintas:

- random split: rendimiento en distribución muy parecida
- split duro: robustez a cambios de día/patrón

La defensa sólida es mostrar ambas, no esconder la más dura.

7. Regla metodológica para el tribunal

Siempre separar:

1. defaults actuales del código
2. configuración real del run histórico citado
3. resultado medido respaldado por artefacto